

```

/**
 *
=====
=====
 * tool-orchestrator-v3-layers.ts
 * طبقات معمارية: Cache + Rate Limiting +
Fallback ذكي + Streaming + Metrics
 * نفس الملف اللي اتفقنا عليه، بدون أي تغيير - شغال)
(فعلياً بمجرد الاستيراد
 *
=====
=====
 */

import { chatWithTools, ToolContext,
TOOL_CAPABLE_MODELS, ChatWithToolsResult }
from "../tool-orchestrator-v2";

// --- 1) Cache ---
interface CacheEntry { value: any;
expiresAt: number; }
const cache = new Map<string, CacheEntry>
();
const DEFAULT_CACHE_TTL_MS = 5 * 60 * 1000;

function cacheKey(userId: string, model:
string, messages: unknown): string {
    return
`${userId}:${model}:${JSON.stringify(messages).slice(0, 500)}`;

```

```

}
export function getCached(userId: string,
model: string, messages: unknown): any |
null {
    const key = cacheKey(userId, model,
messages);
    const entry = cache.get(key);
    if (!entry) return null;
    if (Date.now() > entry.expiresAt) {
cache.delete(key); return null; }
    return entry.value;
}
export function setCached(userId: string,
model: string, messages: unknown, value:
any, ttlMs = DEFAULT_CACHE_TTL_MS) {
    const key = cacheKey(userId, model,
messages);
    cache.set(key, { value, expiresAt:
Date.now() + ttlMs });
}
setInterval(() => {
    const now = Date.now();
    for (const [key, entry] of
cache.entries()) if (now > entry.expiresAt)
cache.delete(key);
}, 10 * 60 * 1000);

// --- 2) Rate Limiting ---
interface RateBucket { count: number;
windowStart: number; }
const rateBuckets = new Map<string,
RateBucket>();
const RATE_LIMIT_PER_MINUTE = 20;
const RATE_WINDOW_MS = 60 * 1000;

```

```

export class RateLimitError extends Error {
  constructor(retryAfterMs: number) {
    super(`تجاوزت الحد المسموح من الطلبات. حاول مرة أخرى بعد ${Math.ceil(retryAfterMs / 1000)} ثانية.`);
    this.name = "RateLimitError";
  }
}

export function checkRateLimit(userId: string) {
  const now = Date.now();
  const bucket = rateBuckets.get(userId);
  if (!bucket || now - bucket.windowStart > RATE_WINDOW_MS) { rateBuckets.set(userId, { count: 1, windowStart: now }); return; }
  if (bucket.count >= RATE_LIMIT_PER_MINUTE) throw new RateLimitError(RATE_WINDOW_MS - (now - bucket.windowStart));
  bucket.count += 1;
}

// --- 3) Fallback ذكي ---
const FALLBACK_CHAIN = [
  "@cf/meta/llama-3.3-70b-instruct-fp8-fast",
  "@cf/meta/llama-3.1-70b-instruct",
  "@cf/meta/llama-3.1-8b-instruct",
  "@cf/openai/gpt-oss-20b",
];

export async function chatWithToolsRobust(
  ctx: ToolContext,

```

```

    preferredModel: string,
    messages: Parameters<typeof
chatWithTools>[2],
    options?: { onToolCall?: (toolName:
string, args: any) => void; useCache?:
boolean }
): Promise<ChatWithToolsResult & {
modelUsed: string; cached: boolean }> {
    checkRateLimit(ctx.userId);

    if (options?.useCache) {
        const cached = getCached(ctx.userId,
preferredModel, messages);
        if (cached) return { ...cached, cached:
true };
    }

    const modelsToTry = [preferredModel,
...FALLBACK_CHAIN.filter((m) => m !==
preferredModel)];
    let lastError: any;
    for (const model of modelsToTry) {
        if
(!TOOL_CAPABLE_MODELS.includes(model))
continue;
        try {
            const result = await
chatWithTools(ctx, model, messages,
options);
            const finalResult = { ...result,
modelUsed: model, cached: false };
            if (options?.useCache)
setCached(ctx.userId, preferredModel,
messages, finalResult);

```

```

        return finalResult;
    } catch (err) {
        lastError = err;
        continue;
    }
}
throw new Error(`فشلت كل النماذج المتاحة في
سلسلة الاحتياط. آخر خطأ
${lastError?.message}
|| "غير معروف"`);
}

// --- 4) Streaming محاكي ---
export async function* streamFinalResponse(
    ctx: ToolContext,
    model: string,
    messages: Parameters<typeof
chatWithTools>[2],
    options?: { onToolCall?: (toolName:
string, args: any) => void; chunkSize?:
number; delayMs?: number }
): AsyncGenerator<{ type: "tool_call" |
"text_chunk" | "done"; data: any }> {
    const result = await
chatWithToolsRobust(ctx, model, messages, {
onToolCall: (t, a) => options?.onToolCall?.
(t, a) });
    for (const call of result.toolCallsLog)
yield { type: "tool_call", data: { tool:
call.tool, durationMs: call.durationMs } };

    const chunkSize = options?.chunkSize ??
24;
    const delayMs = options?.delayMs ?? 20;
    const text = result.finalMessage;

```

```
    for (let i = 0; i < text.length; i +=
chunkSize) {
        yield { type: "text_chunk", data:
text.slice(i, i + chunkSize) };
        if (delayMs > 0) await new Promise((r)
=> setTimeout(r, delayMs));
    }
    yield { type: "done", data: { modelUsed:
result.modelUsed, iterations:
result.iterations } };
}

// --- 5) Metrics ---
export function getSystemMetrics() {
    return { cacheSize: cache.size,
activeRateBuckets: rateBuckets.size,
rateLimitPerMinute: RATE_LIMIT_PER_MINUTE,
fallbackChain: FALLBACK_CHAIN };
}
```